

HACKATHON RAPTORS • DOGFOOD 2026

Dogfood 2026

Implementation Plan

Hackathon Submission & Judging Platform

TEAM CODEHAWK

Prepared for: Team CodeHawk | Dogfood 2026 (Hackathon Raptors)

Format: 72-hour build, tracked in elapsed hours from kickoff (Hour 0) to submission (Hour 72)

Contents

1. Executive Strategy	1
2. Team Role Assignment (4-member baseline)	2
3. Tech Stack Recommendation	3
4. System Architecture	5
5. Data Model	6
6. Tier-by-Tier Build Plan	7
T1 — Core	7
T2 — Judging	7
T3 — Public	8
T4 — Stretch	8
7. Hour-by-Hour 72-Hour Timeline	8
8. Judging Integrity Design	9
Judge Assignment Algorithm	9
Cross-Judge Score Normalization	10
Role Isolation Enforcement	10
9. Anti-Abuse and Voting Design (T3)	11
10. Bonus Challenge Prioritization	12
11. Risk Register	14
12. Final Submission Checklist	16

Team: CodeHawk, 4 members. Section 2 assigns a fourth, dedicated role — splitting judging/scoring ownership away from the rest of the backend — and includes a fallback for teams of 1-3 in case anyone drops before kickoff.

Companion files. This document is the human-facing strategy plan. A separate **PLAN.md** is the execution-ready companion built for Claude Code (or any AI coding assistant) to follow phase-by-phase (Phase 0-5, one phase per tier plus a bootstrap and a final polish pass) during the 72 hours — it mirrors this plan's tiers and architecture but in agent-actionable form (file paths, exact commands, checklists), and adds a dedicated user-experience checklist to every phase (loading/empty/error states, inline form validation, responsive breakpoints, keyboard navigation) so usability is a gate at each step, not an afterthought. Two more files you supply will slot into that build: **DESIGN_SYSTEM.md** (colors, typography, spacing, and component conventions for consistency across the whole app) and **reference_design.pdf** (the target visual design for the frontend landing page). **PLAN.md** instructs Claude Code to treat both as the source of truth for anything visual.

1. Executive Strategy

What “done” means, checkpoint by checkpoint:

Checkpoint	Time (from kickoff)	“Done” bar
Scaffold live	Hour 8	<code>docker compose up</code> boots empty DB + empty API + empty frontend shell, all three containers healthy, zero manual steps
T1 pass	Hour 30	Acceptance suite's T1 checks green: auth, 4 roles, event CRUD, team invites, submission draft/edit, deadline enforcement, public gallery search
T2 pass	Hour 50	Judge assignment, rubrics, role isolation, progress dashboard, normalization, CSV export all pass acceptance suite
T3 pass (stretch-but-targeted)	Hour 62	Voting, comments, hidden-results, ordering randomization, rate limiting, duplicate detection, audit trail
Freeze prep	Hour 68	Feature freeze. Only bugfixes, docs, demo video, acceptance report from here
Submission	Hour 72	Repo public, license in place, <code>acceptance-report.txt</code> generated, all four <code>.md</code> docs complete, demo video uploaded

Sequencing logic: The judging weights (40% tier completion, 25% judging integrity, 20% adoptability, 15% quality) mean **T2 is worth more than T3 or T4 combined** once you weigh “judging integrity” against them — a broken T3 costs you nothing extra if T2 is airtight, but a broken T2 tanks 25% of your score directly and drags tier-completion down too. Therefore:

- T1 and T2 are non-negotiable, done with margin, and validated against the acceptance suite *before* touching T3.
- T3 is attempted only after T2 is verified green — if you’re behind schedule at Hour 50, cut T3 scope (pick the “alternative voting mechanism” defensible fallback, see Section 9) rather than rushing T2.
- T4 and bonus challenges are opportunistic: attempted only in the last 10–14 hours, and only the single highest-leverage bonus (see Section 10), never at the cost of documentation or the acceptance report.
- Documentation is **not** a Hour-70 activity. It is written incrementally as each tier passes, by whichever teammate isn’t heads-down in code at that moment (see Section 2). This is the single biggest lever against the “ran out of time to document” failure mode that separates the acceptance-suite-verified submissions from the disqualified ones.

Core principle for the whole 72 hours: correctness and defensibility over feature count. A team that ships a clean T2 with a rigorous JUDGING.md beats a team that claims T4 with an unverifiable normalization method.

2. Team Role Assignment (4-member baseline)

Role	Owns	Time split
Auth & Roles Lead	User model, password hashing, session auth, role enum, RBAC middleware, team model + invite-link logic, deadline enforcement	~85% code, 15% docs (DATA-MODEL.md auth/team sections)
Judging & Scoring Lead	Rubric CRUD, judge assignment algorithm, score submission endpoint, normalization pipeline, judging-side role-isolation tests	~85% code, 15% docs (JUDGING.md, normalization math)
Frontend/Product Lead	Event creation UI, team formation, submission flow, public gallery, judge dashboards, voting UI	~85% code, 15% docs (README.md user-facing sections)
Platform/Integration Lead	Docker Compose, seed fixtures, CSV export, acceptance suite runs, CI-equivalent local test harness, demo video, ARCHITECTURE.md, coordinates freeze	~55% code (glue + tests), 45% docs/ops — this role is deliberately docs-heavy because someone has to own the non-code deliverables continuously, not at Hour 70

Standups: 15 minutes at the start of every 8-hour block (see Section 7), synced to the

acceptance-suite checkpoint schedule. Splitting Auth & Roles from Judging & Scoring (rather than the more common 3-role split) is what gives this team enough spare bandwidth to attempt a T4 item and a bonus challenge (Section 10) without putting T1-T3 at risk — the two backend-heavy roles can run their acceptance-suite gates in parallel instead of serially.

Fallback plans (if a teammate drops before kickoff):

- **Solo (1 member):** Compress to T1 + T2 only, treat T3 as a stretch you attempt only if T2 passes acceptance by Hour 40. Skip T4 and all bonus challenges entirely — spend saved time on documentation and a rock-solid demo video, since Adoptability (20%) and Judging Integrity (25%) are achievable solo but breadth is not.
- **2 members:** One person covers Auth & Roles Lead + Judging & Scoring Lead (both backend-heavy, and the acceptance-suite gates for T1/T2 naturally sequence one after the other anyway); the other covers Frontend/Product Lead + Platform/Integration Lead. Target T1 + T2 fully, attempt T3's simplest defensible option (see Section 9) only.
- **3 members:** Merge Judging & Scoring Lead back into Auth & Roles Lead as a single combined Backend/Data Lead; keep Frontend/Product Lead and Platform/Integration Lead as separate owners. This is the leanest configuration that can still reliably clear T1-T3; treat T4 and bonus challenges as opportunistic only, per Section 10.

3. Tech Stack Recommendation

Layer	Choice	Why
Backend	Python 3.12 + FastAPI	Your team's strongest language; FastAPI gives you Pydantic validation, automatic OpenAPI docs (free T4 credit if you attempt API-First), and async support without extra ceremony
ORM	SQLModel (SQLAlchemy 2.0 + Pydantic under one roof)	One model definition serves both DB schema and API schemas — cuts boilerplate roughly in half versus separate SQLAlchemy models + Pydantic schemas, which matters when every hour counts
Database	PostgreSQL 16 , containerized, data volume mounted locally	Real relational integrity (FKs, constraints) for judging correctness; still starts with zero config via Compose; avoids SQLite's weak concurrent-write story if judges are scoring simultaneously

Layer	Choice	Why
Auth	Session cookies signed with <code>itsdangerous</code> , passwords hashed with <code>passlib[bcrypt]</code> , no JWT-as-a-service, no OAuth provider	Fully offline, no external identity provider, trivial to seed fixture users into at boot
Frontend	React + TypeScript + Vite , TailwindCSS	Fast scaffold, your team already knows TypeScript, Vite's dev server and build are near-instant, Tailwind avoids time lost hand-rolling CSS
Containerization	Docker Compose , 3 services: <code>db</code> (postgres), <code>api</code> (FastAPI, serves built frontend as static files in prod mode), <code>web</code> (Vite dev server, optional — collapse into <code>api</code> for the actual submission so it's truly one command with no dev-only dependency)	Single command startup is a hard constraint; serving the built SPA from FastAPI's <code>StaticFiles</code> means the submission only truly needs <code>db</code> + <code>api</code> containers, minimizing failure surface
Testing	pytest (backend, incl. <code>pytest-asyncio</code> + <code>httpx</code> test client) and Vitest (frontend unit) + Playwright (one end-to-end lifecycle test matching the demo video script)	Playwright's single "full lifecycle" test doubles as your demo-video rehearsal script
CSV export	Python <code>csv</code> <code>stdlib</code>	Zero dependency risk, zero network calls, sufficient for T2's requirement

Alternative stack considered and rejected: Next.js (App Router) + Prisma + SQLite, single container.

- *Pros*: Genuinely faster to scaffold a CRUD-heavy app solo; Prisma's schema DSL and migrations are ergonomic; one language (TypeScript) end to end reduces context-switching.
- *Why rejected*: (1) Your team's stated comfort level is strongest in Python/TypeScript combined, not TypeScript-only, so you're not capturing the "one language" benefit as much as a TS-native team would. (2) SQLite under concurrent judge-scoring writes (multiple judges submitting scores near-simultaneously during the judging window) is more fragile than Postgres's proper row-level locking and constraint enforcement, which is directly relevant to Judging Integrity (25% of your score). (3) Prisma's migration engine occasionally needs network access to fetch its query-engine binary on `npm install` in fresh environments unless you vendor it carefully — a real risk against "must run on a laptop with the network turned off," whereas `psycopg2`/SQLAlchemy wheels are simpler to guarantee offline via `pip download` pre-kickoff. (4) FastAPI's automatic OpenAPI generation is a bigger unlock for the API-First bonus challenge than Next.js API routes, which need

manual OpenAPI authoring.

4. System Architecture

Pattern: Modular monolith, not microservices. Justification: microservices multiply the number of things that must all start correctly under `docker compose up`, multiply inter-service network calls (which must all stay on the Compose-internal network, adding failure modes for zero benefit at this scale), and buy you nothing — there’s one team, one deploy target, one dataset. A modular monolith with clean internal module boundaries (auth, events, teams, submissions, judging, scoring, voting, export) gets you the same separation-of-concerns benefit for Code Quality (15%) without the operational risk that directly threatens the Adoptability criterion (20%) and the hard “no network connectivity” constraint.

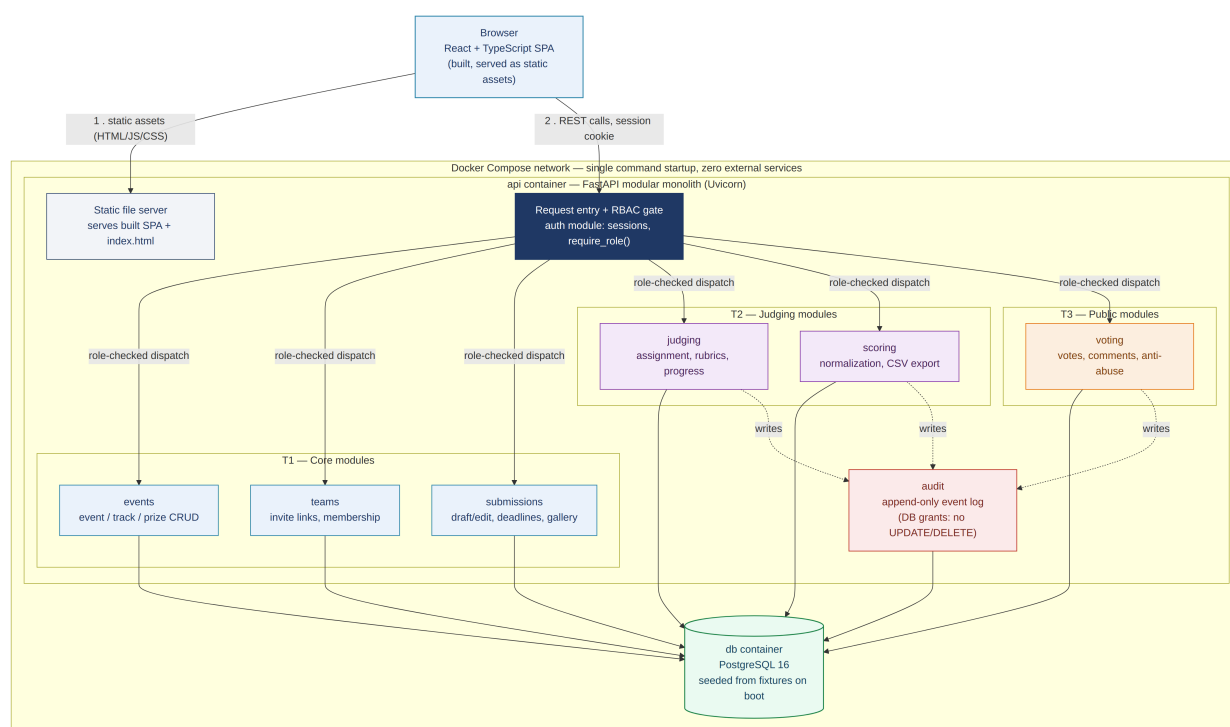


Figure 1: System architecture diagram — request flow, module boundaries by tier, and audit-log write paths

Color key: navy = the single RBAC gate every request passes through; blue = T1 core modules; purple = T2 judging modules; orange = T3 public/voting modules; red = the append-only audit log; green = the seeded Postgres container. Dotted arrows are audit writes — every judging, scoring, and voting action is logged there in addition to its own table.

Module boundaries are enforced at the code level (separate FastAPI `APIRouters`, separate service classes, no direct cross-module DB queries — go through the owning module’s service layer) so role-isolation logic lives in exactly one place (`auth` module’s dependency-injected `require_role()` guard) rather than being re-implemented per endpoint.

5. Data Model

Entity	Key fields	Relationships	Import/export notes
User	id, email, password_hash, role (enum: participant/judge/organizer/admin), created_at	1—N TeamMembership, 1—N JudgeAssignment	CSV export of user list (organizer/admin only)
Event	id, name, slug, start_at, end_at, tracks (JSON or Track table), prize_config (JSON), voting_enabled, results_hidden_until	1—N Team, 1—N Submission, 1—N Rubric	Export path: full event archive (JSON) for T4 bulk export
Team	id, event_id, name, invite_token, invite_expires_at	N—N User via TeamMembership, 1—N Submission	Invite token is the import path for joining — no email service needed
TeamMembership	user_id, team_id, joined_at	Join table	—
Submission	id, team_id, event_id, title, description, repo_url, status (draft/submitted), submitted_at, track_id	1—N JudgeAssignment, 1—N Vote, 1—N Comment	CSV export of gallery; JSON export for T4
Rubric	id, event_id, criteria (JSON: [{name, weight, max_score}])	belongs to Event, referenced by Score	Weights must sum to 1.0 — validate on save
JudgeAssignment	id, judge_id, submission_id, assigned_at, status (pending/in_progress/completed)	belongs to User(judge), Submission	Drives progress dashboard; CSV export of assignment matrix
Score	id, judge_assignment_id, criterion_scores (JSON matching Rubric.criteria), raw_total, submitted_at	belongs to JudgeAssignment	Feeds the normalization pipeline (Section 8); never mutated after submission — corrections create a new versioned row, old row flagged superseded, both kept for audit
Vote	id, user_id, submission_id, created_at, fingerprint_hash	Unique constraint (user_id, submission_id) prevents double-voting	Duplicate-detection also checks fingerprint_hash (see Section 9)

Entity	Key fields	Relationships	Import/export notes
Comment	id, user_id, submission_id, body, created_at	belongs to User, Submission	—
AuditLog	id, actor_id, action, target_type, target_id, metadata (JSON), created_at	Append-only, no updates/deletes permitted at the DB layer (revoke UPDATE/DELETE grants on this table in migration)	This is your auditability story for JUDGING.md — every score submission, assignment change, and vote is a row here

Import/export paths that need explicit code: (1) fixture seed script reads JSON/CSV fixtures on container boot, idempotent (checks if already seeded), (2) CSV export endpoints for users/submissions/scores/assignments (T2 requirement), (3) full JSON event export/import for T4 bulk import/export bonus scope.

6. Tier-by-Tier Build Plan

T1 — Core

Build order: (1) User model + password hashing + session auth → (2) role enum + `require_role()` middleware → (3) Event CRUD (organizer/admin only) → (4) Team model + invite-link generation/redemption → (5) Submission draft/edit with autosave-on-blur → (6) deadline enforcement (server-side check against `Event.end_at`, reject writes after) → (7) public gallery with search (simple `ILIKE` on title/description is sufficient — do not build full-text search infra here).

Hidden complexity / pitfalls

Deadline enforcement is commonly checked only in the frontend (disabled button) and forgotten server-side — the acceptance suite *will* hit the endpoint directly after the deadline. Invite-link expiry needs a server-side check too, not just a UI hide. Role enum must be enforced at every mutating endpoint, not just page-level routes — a common gap is forgetting PATCH/DELETE routes when RBAC was only added to GET/POST.

T2 — Judging

Build order: (1) Rubric CRUD (organizer) → (2) JudgeAssignment model + assignment algorithm (Section 8) → (3) Score submission endpoint restricted to assigned judge only → (4) role-isolation tests (a judge cannot see other judges' scores, a participant cannot see any scores, an organizer can see aggregate but the acceptance suite may check they can't silently edit raw scores) → (5) judge progress dashboard (assigned count vs. completed count) → (6) normalization pipeline (Section 8) → (7) CSV export for scores, assignments, normalized results.

Hidden complexity / pitfalls

Normalization must run *after* all assigned judges for a submission have scored, or on-demand recompute — decide explicitly and document it (partial-data normalization looks plausible but is a common defensibility gap in JUDGING.md if not addressed). Role isolation for organizers is

subtler than judges/participants: organizers typically need to see everything but the acceptance suite may specifically test that they cannot masquerade as a judge to submit a score — enforce at the endpoint, not just the UI.

T3 — Public

Build order: (1) voting toggle per event (`Event.voting_enabled`) → (2) hidden-results flag tied to `Event.results_hidden_until` → (3) randomized project ordering (seed the shuffle per-session so it's stable within a browsing session but random across users — avoids the “always shows the same top project” bias complaint) → (4) comments → (5) rate limiting → (6) duplicate detection → (7) audit trail (mostly already covered by the `AuditLog` table from Section 5 — wire remaining actions into it here).

Hidden complexity / pitfalls

“Hidden results during voting” is easy to get wrong by hiding results in the UI while still leaking the raw score/vote count through an unauthenticated API response — the acceptance suite is likely to probe the API directly, not just the rendered page.

T4 — Stretch

Only attempt if T1–T3 are acceptance-suite green with time remaining. Priority order if attempted: (1) REST API + OpenAPI docs (FastAPI gives you this close to free if your route design was clean) → (2) certificate/record generation (server-side PDF or HTML-to-print, simple template, no external service) → (3) bulk import/export → (4) signed judge participation records (requires a local keypair — do not reach for an external signing service, use `cryptography`'s `ed25519` support entirely offline) → (5) embeddable gallery widget (lowest priority — cut first if time runs short).

7. Hour-by-Hour 72-Hour Timeline

All times below are hours elapsed since kickoff (**Hour 0**), running through the **72-hour** window to submission (**Hour 72**).

Hour Block	Focus	Checkpoint gate
Hour 0–8	Repo init (code only after kickoff — see rules), Docker Compose skeleton, DB schema migration for T1 entities, auth scaffolding	Gate: <code>docker compose up</code> boots all containers healthy, empty DB reachable
Hour 8–16	Finish T1: roles, event CRUD, team invites, submission draft/edit	—
Hour 16–24	Finish T1: deadline enforcement, public gallery + search; run acceptance suite against T1	Gate: T1 acceptance suite green before starting T2
Hour 24–32	T2 start: rubric CRUD, judge assignment algorithm, score submission endpoint	—

Hour Block	Focus	Checkpoint gate
Hour 32-40	T2: role-isolation hardening + tests, progress dashboard, normalization pipeline	—
Hour 40-50	T2: CSV exports, finish normalization edge cases (missing scores, single-judge submissions); run acceptance suite against T2	Gate: T2 acceptance suite green before starting T3
Hour 50-58	T3: voting, hidden results, ordering randomization, comments	—
Hour 58-62	T3: rate limiting, duplicate detection, audit trail wiring; run acceptance suite against T3	Gate: T3 pass or explicit documented fallback (Section 9) — do not leave half-built
Hour 62-68	T4/bonus window only if all above gates are green — pick the single highest-leverage item from Section 10; otherwise this block becomes buffer + polish	—
Hour 68-70	Feature freeze. Final acceptance suite full run, generate <code>acceptance-report.txt</code> , fix only what the report flags	Gate: acceptance report generated and committed
Hour 70-71	Record 5-minute demo video (script = the Playwright end-to-end test path), finalize README/ARCHITECTURE/DAMODEL/JUDGING docs	—
Hour 71-72	Final submission checklist (Section 12), make repo public, submit	Hard deadline: Hour 72

8. Judging Integrity Design

Judge Assignment Algorithm

Balanced conflict-aware round-robin assignment. For an event with J judges and S submissions, and a configured `assignments_per_submission = k` (organizer-configurable, default 3):

1. Build a conflict list: exclude any (judge, submission) pair where the judge is a member of the submitting team (self-review prevention), sourced directly from `TeamMembership`.
2. Sort submissions by track (to encourage track-relevant judges to see track-relevant work, if judges are optionally tagged with track expertise — optional refinement, skip if time-

pressed).

3. Greedily assign: iterate submissions in round-robin order, for each pick the k judges with (a) no conflict and (b) the fewest current assignments so far, ties broken by judge ID for determinism (determinism matters for reproducibility and defensibility in JUDGING.md).
4. This is a degree-constrained bipartite assignment solved greedily rather than via a full min-cost-flow solver — document this explicitly as a deliberate simplicity-over-optimality trade-off appropriate for hackathon-scale data (dozens to low hundreds of submissions), and note that a max-flow formulation (Hopcroft-Karp or a min-cost flow with capacity constraints) is the documented upgrade path if Hackathon Raptors' real event scale requires perfect load-balancing.

Cross-Judge Score Normalization

Judges differ in leniency and spread. Use **per-judge z-score normalization**, then map back to a display scale:

For judge j who has submitted scores for a set of submissions, with raw total scores $x_{j,1}, \dots, x_{j,n}$:

$$\mu_j = \frac{1}{n} \sum_{i=1}^n x_{j,i} \quad \sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_{j,i} - \mu_j)^2}$$

$$z_{j,i} = \frac{x_{j,i} - \mu_j}{\sigma_j}$$

If $\sigma_j = 0$, set $z_{j,i} = 0$ for all i from that judge — a judge who gave every submission the same score contributes no discriminating signal.

For a submission i scored by judge set J_i :

$$\bar{z}_i = \frac{1}{|J_i|} \sum_{j \in J_i} z_{j,i}$$

Map back to a 0-100 display scale for readability using a fixed linear transform (e.g., $display_i = 50 + 10 \cdot \bar{z}_i$, clamped to [0,100]) — document the clamp explicitly since z-scores are unbounded in theory.

Why this method, for JUDGING.md: it corrects for judges who are systematically harsher or more lenient (mean-centering) and for judges who use a wider or narrower range of scores (scaling by spread), without requiring any assumption about the true underlying quality distribution beyond per-judge approximate normality — which is reasonable at hackathon-scale rubric scoring. This directly maps to the **Normalization Proof bonus challenge** (Section 10): show the raw vs. normalized rankings side-by-side on the fixture data to demonstrate the effect concretely.

Role Isolation Enforcement

Enforce at a single choke point: a FastAPI dependency `require_role(*allowed_roles)` injected into every mutating and every sensitive-read endpoint, backed by the session's stored role — never trust a role claim from the request body. Additionally:

- Judges can only fetch/submit scores for `JudgeAssignment` rows where the assignment's judge matches the logged-in user (`assignment.judge_id == current_user.id`).

- Participants can never fetch score detail for any submission (only their own team's, and only after `results_hidden_until` if voting is enabled).
- Every role-check has a corresponding pytest that asserts a 403 for each *other* role attempting the action — this test matrix is your Judging Integrity defensibility evidence, and its existence should be explicitly cited in JUDGING.md.

9. Anti-Abuse and Voting Design (T3)

Mechanism	Design	Reasoning
Rate limiting	Simple in-process token-bucket per (user_id, IP) pair on vote/comment endpoints (e.g., <code>slowapi</code> or a hand-rolled Redis-free in-memory limiter keyed by session, since no external service is allowed) — cap votes per minute and total votes per event	Prevents scripted rapid-fire voting without needing an external rate-limiting service, keeping the “no network connectivity” constraint intact
Duplicate detection	DB-level unique constraint on (user_id, submission_id) in <code>Vote</code> as the primary guard, plus a <code>fingerprint_hash</code> (hash of session metadata such as user-agent + a server-set random cookie set at first visit) checked for suspiciously repeated fingerprints across different accounts voting for the same submission in a short window	Unique constraint stops the trivial case (same account voting twice); fingerprinting is a documented, honest best-effort layer against multi-account abuse — do not oversell this in JUDGING.md, frame it explicitly as a deterrent, not a guarantee
Audit trails	Every vote, comment, and score change writes an <code>AuditLog</code> row with actor, action, target, timestamp, and metadata; the table has DB-level <code>REVOKE UPDATE, DELETE</code> so it is append-only even against a compromised app-layer bug	Auditability is explicitly named in the Judging Integrity criterion — an append-only log enforced at the DB grant level (not just “we don’t call UPDATE in our code”) is a materially stronger, more defensible claim

Mechanism	Design	Reasoning
Alternative to full voting (fallback if time-pressed)	If T3's full voting stack risks not finishing cleanly, implement the "technically-defended alternative" the spec explicitly allows: judge-only scoring with a lightweight, capped "community interest" signal (one thumbs-up per authenticated user, no ranking impact, purely informational) — much smaller surface area than full voting + comments + ordering randomization, but still satisfies the T3 allowance	The spec explicitly permits this trade — a small, clean, honestly-scoped T3 beats a half-built full voting system when verified against the acceptance suite

10. Bonus Challenge Prioritization

Bonus	Effort (for this stack)	Payoff	Rank	Recommendation
Normalization Proof (+5)	Low — the z-score pipeline from Section 8 already exists for T2; this bonus is mostly presenting a before/after table on fixture data	High	1st	Attempt this first, in the T4/bonus window (Hour 62-68). Near-zero marginal engineering cost since T2 already requires the machinery.

Bonus	Effort (for this stack)	Payoff	Rank	Recommendation
Threat Model (+3)	Low — a focused write-up (Sybil voting, ballot stuffing, submission scraping, judge collusion, deadline gaming), each with the mitigation you already built in Sections 8–9	Medium	2nd	Attempt second. This is primarily a documentation task once T2/T3 mitigations exist — cheap to write, and it strengthens JUDGING.md’s defensibility narrative regardless of whether you claim the bonus points.
API First (+3)	Medium — FastAPI’s auto-generated OpenAPI spec covers most of this for free, but “every UI action” requires an audit pass to ensure no action is UI-only (e.g., invite-link redemption might be a raw DB write triggered from a link click rather than a documented endpoint)	Medium	3rd	With the 4-person split from Section 2 (dedicated Judging & Scoring Lead freeing the rest of the team), this is realistic to attempt in the Hour 62–68 window if the T1–T3 gates are green; drop it first if the Hour 62 checkpoint shows no slack.

Bonus	Effort (for this stack)	Payoff	Rank	Recommendation
Pairwise Mode / Bradley-Terry (+5)	High — requires a separate comparison data model, a pairwise UI, and implementing/-validating a Bradley-Terry MLE estimator (or justified alternative) correctly enough to be “defensible,” which is easy to get subtly wrong under time pressure	High, but high risk	4th — deprioritize	Do not attempt unless T1-T3 are green with 10+ hours of genuine slack remaining and a team member is confident implementing and <i>validating</i> the estimator (e.g., against a known small worked example) in under 4 hours. A broken pairwise system is worse for Code Quality & Innovation (15%) than not attempting it at all.

Overall guidance: attempt at most Normalization Proof + Threat Model as your default bonus scope (8 combined points, low risk to core tiers); treat the other two as pure upside only if the Hour 62 gate shows real slack.

11. Risk Register

#	Risk	Mitigation
1	Hidden network dependency surfaces at the worst time (an <code>npm install</code> or <code>pip install</code> reaches out mid-hackathon, or a package’s postinstall script phones home)	Pre-kickoff (allowed: planning/research, not code): identify exact package versions, run <code>pip download / npm pack</code> for your intended dependency set in a network-off VM <i>before</i> kickoff to confirm they install clean offline; pin exact versions in <code>requirements.txt/package-lock.json</code> day one

#	Risk	Mitigation
2	Docker build fails or is slow on judges' machines due to base image size or build context bloat	Use slim base images (<code>python:3.12-slim</code> , <code>node:20-slim</code> for the build stage only, multi-stage build so the final image doesn't ship Node), add a <code>.dockerignore</code> , test the full <code>docker compose up</code> on a clean machine/VM at least twice before submission, not just on the dev machine with cached layers
3	Normalization logic works but the team can't clearly explain/defend it under judge questioning	Write the JUDGING.md math section (Section 8 of this plan, essentially verbatim) <i>as you build it</i> , not after; include the worked fixture-data example required for the Normalization Proof bonus regardless of whether you claim the bonus — it doubles as your defensibility artifact
4	Documentation (<code>README.md</code> , <code>ARCHITECTURE.md</code> , <code>DATA-MODEL.md</code> , <code>JUDGING.md</code>) gets squeezed to the last few hours and comes out thin	Platform/Integration Lead (Section 2) owns docs continuously across all blocks, not just at freeze; each tier gate in Section 7 includes "update relevant doc section" as part of the gate, not a separate task
5	Role-isolation bug slips through (a judge can see another judge's scores, or a participant can hit a privileged endpoint directly)	The pytest matrix described in Section 8 (every endpoint $\times$ every disallowed role $\rightarrow$ <code>assert 403</code>) is written <i>alongside</i> each endpoint, not as a separate late-stage QA pass; run it as a gate before every tier's acceptance-suite run
6	Judge assignment algorithm produces an unbalanced or conflicted assignment on the actual fixture data (more judges assigned to some submissions than others, or a conflict slips through)	Unit test the assignment function directly against the provided fixture data as soon as it's published — before writing any code, sketch the expected assignment matrix by hand so you have a fast sanity check the moment code exists

#	Risk	Mitigation
7	Deadline enforcement or hidden-results logic is checked in the UI but not the API, and the acceptance suite calls the API directly	Treat “server-side enforcement, UI enforcement is a UX nicety only” as a standing rule, not a per-feature decision — state this explicitly at kickoff standup so no teammate defaults to a frontend-only check under time pressure
8	Demo video recording runs long or requires re-takes, eating into the final submission window	Script the demo video directly from the Playwright end-to-end test path (Section 3) well before Hour 70, do a dry run at the T3 gate (~Hour 62) even with placeholder data, so the Hour 70–71 block is a recording of a rehearsed path, not an improvisation

12. Final Submission Checklist

Deliverable	Checklist item	What “acceptance-suite-verified” means here
Public GitHub repo	Repository visibility set to public <i>before</i> the deadline	N/A — verified by organizers checking repo visibility directly, not the acceptance suite
OSI-approved license	<code>LICENSE</code> file present (e.g., MIT or Apache-2.0) at repo root	N/A — manual/organizer check
<code>docker compose up</code>	One command, no flags, no manual env-var setup beyond what’s committed (with safe non-secret defaults), boots a seeded working portal	The acceptance suite’s first action is almost certainly running exactly this command against a clean checkout — test on a machine that has never had your repo before
<code>acceptance-report.txt</code>	Generated by running the provided acceptance suite locally against your build, output saved verbatim, committed to repo root	This is the artifact the suite’s own report is compared against — do not hand-edit it; if a check fails, report it honestly rather than omitting it (explicitly rewarded per judging criteria)

Deliverable	Checklist item	What “acceptance-suite-verified” means here
README.md	Setup instructions, feature overview, tier claims stated plainly and matched to acceptance report	Cross-checked against acceptance-report.txt — a tier claim in the README not backed by the report is a credibility risk against “honest gap reporting is rewarded”
ARCHITECTURE.md	System design (can reuse Section 4’s Mermaid diagram and reasoning), major technical decisions and why	No suite check, but directly scored under Code Quality & Innovation and Adoptability
DATA-MODEL.md	Schema documentation (can reuse Section 5’s table), import/export paths noted explicitly	No suite check, but explicitly required and scored under Adoptability
JUDGING.md	Assignment algorithm, scoring methodology, normalization method with the math (Section 8), reasoning	This is your primary Judging Integrity (25%) evidence document — treat it as load-bearing, not boilerplate
Tests	Backend pytest suite, frontend Vitest unit tests, at least one Playwright end-to-end lifecycle test	The acceptance suite likely checks tier <i>behavior</i> , not your test suite directly — but your own tests are what stop you from failing the acceptance suite in the first place, and are scored under Code Quality
5-minute demo video	Shows one complete event lifecycle: event creation → team formation → submission → judge assignment → scoring → normalized results → (if built) voting	No automated check — this is a human-judged artifact, keep it tightly scripted to the rehearsed Playwright path from Section 11, item 8
Tier claims honesty	Every tier claimed in README matches what acceptance-report.txt shows as passing	Directly scored: “Correctness matters more than feature count, and honest gap reporting is rewarded”

End of implementation plan. Good luck at kickoff.